

Introduction to Programming: Python – Course 1

Mengzhuo Guo

Academic Year 2026-2027

Contents

1	Conda Environment	2
1.1	What Is Conda?	2
1.2	Opening the Command-Line Interface	2
1.3	Checking the Conda Installation	2
1.4	Getting Help	3
1.5	Updating Conda	3
1.6	Understanding the Base Environment	3
1.7	Creating a Virtual Environment	4
1.8	Listing Existing Environments	4
1.9	Activating an Environment	5
1.10	Deactivating an Environment	5
1.11	Installing Packages	5
1.12	Installing a Specific Package Version	6
1.13	Installing Packages from a Channel	6
1.14	Searching for a Package	6
1.15	Viewing Installed Packages	6
1.16	Testing an Installed Package	7
1.17	Updating Packages	7
1.18	Removing Packages	7
1.19	Using Pip inside a Conda Environment	8
1.20	Installing and Starting JupyterLab	8
1.21	Registering a Jupyter Kernel	8
1.22	Cloning an Environment	9
1.23	Exporting an Environment	9
1.24	Creating an Environment from a YAML File	9
1.25	Removing an Environment	9
1.26	Cleaning the Conda Cache	10
1.27	Controlling Automatic Activation of Base	10
1.28	Initializing Conda for a Terminal	10
1.29	Common Problems	11
1.30	Recommended Workflow for This Course	12
1.31	Summary of Common Conda Commands	13
1.32	Important Reminders	13
2	Assignment: Conda Environment Practice	14
2.1	Objectives	14
2.2	Submission Requirements	14
2.3	Task 1: Install Anaconda and Create an Environment	14
2.4	Task 2: Display Personal and Environment Information	15
2.5	Task 3: Check the Python Interpreter	15
2.6	Task 4: Install and Test Python Packages	16
2.7	Task 5: Create and Run a Python File	16
2.8	Task 6: Export the Conda Environment	17
2.9	Task 7: Deactivate and Reactivate the Environment	17

2.10 Optional Challenge: Compare Two Environments	18
2.11 Checklist	18
2.12 Grading Criteria	19
2.13 Important Notes	19

1 Conda Environment

1.1 What Is Conda?

Conda is an environment manager and a package manager. It can be used to:

- create independent Python environments;
- install different versions of Python;
- install, update, and remove Python packages;
- avoid package conflicts between different projects;
- reproduce and share a programming environment.

A Conda environment is an isolated directory containing a particular version of Python and a collection of installed packages. Packages installed in one environment normally do not affect other environments.

For example, one project may use Python 3.12, while another project may use a different version of Python.

1.2 Opening the Command-Line Interface

Conda commands should be entered in a command-line interface.

- **Windows:** Open **Anaconda Prompt** from the Start menu.
- **macOS:** Open the Terminal application.
- **Linux:** Open a terminal window.

For Windows beginners, using Anaconda Prompt is recommended because Conda is usually initialized automatically.

A command is entered after the prompt symbol. For example:

```
(base) C:\Users\username> conda --version
```

The text **(base)** indicates that the default Conda environment is currently active.

1.3 Checking the Conda Installation

After installing Anaconda or Miniconda, use the following command to check whether Conda is available:

```
conda --version
```

An example output is:

```
conda 25.5.1
```

The exact version number may be different. If a version number is displayed, Conda has normally been installed successfully.

The following command displays more detailed information about Conda:

```
conda info
```

It shows information such as:

- the current active environment;
- the Conda installation location;
- the Python version used by Conda;
- the operating system;
- the configured package channels;
- the directories used to store environments.

The location of the Conda executable can also be checked.

On Windows:

where conda

On macOS or Linux:

which conda

To check the current Python version, run:

```
python --version
```

To check the location of the Python interpreter currently being used, run:

```
python -c "import sys; print(sys.executable)"
```

This command is particularly useful when several Python environments are installed on the same computer.

1.4 Getting Help

Most Conda commands provide a help page through the `--help` option.

For example:

```
conda --help
conda create --help
conda install --help
conda remove --help
```

When you are unsure about the options of a command, use `--help` before searching for other solutions.

1.5 Updating Conda

Conda itself is normally managed in the **base** environment. To update Conda, run:

```
conda update -n base conda
```

Conda will display the packages that will be updated and ask for confirmation:

```
Proceed ([y]/n)?
```

Enter **y** and press Enter to continue.

1.6 Understanding the Base Environment

After opening Anaconda Prompt, the command line may begin with:

```
(base)
```

The **base** environment is the default environment created during the Conda installation.

It is recommended that students do not install all course packages directly into the **base** environment. Instead, create a separate environment for each course or project. This reduces dependency conflicts and makes the environment easier to reproduce.

1.7 Creating a Virtual Environment

The general syntax for creating a Conda environment is:

```
conda create --name environment-name python=version
```

The shorter option `-n` is equivalent to `--name`.

For this course, an environment named `python-course` can be created as follows:

```
conda create --name python-course python=3.12
```

The shorter version is:

```
conda create -n python-course python=3.12
```

In this command:

- `conda create` creates a new environment;
- `-n python-course` specifies the environment name;
- `python=3.12` specifies the Python version.

When Conda asks whether to proceed, enter:

y

It is also possible to create an environment and install several packages at the same time:

```
conda create -n python-course python=3.12 numpy pandas matplotlib
```

Environment names should be short and meaningful. Avoid spaces and special characters in environment names.

1.8 Listing Existing Environments

To display all Conda environments, run:

```
conda env list
```

An equivalent command is:

```
conda info --envs
```

An example output is:

```
# conda environments:
#
base                *  D:\anaconda3
python-course       D:\anaconda3\envs\python-course
```

The asterisk `*` indicates the currently active environment.

1.9 Activating an Environment

Before using a Conda environment, it should be activated.

To activate the course environment, run:

```
conda activate python-course
```

After activation, the command prompt changes from:

```
(base)
```

to:

```
(python-course)
```

This indicates that commands such as `python`, `pip`, and `jupyter` will use the programs installed in the `python-course` environment.

After activating the environment, verify the Python version:

```
python --version
```

You can also verify the location of the Python interpreter:

```
python -c "import sys; print(sys.executable)"
```

1.10 Deactivating an Environment

To leave the current environment, run:

```
conda deactivate
```

For example, the command prompt may change from:

```
(python-course)
```

back to:

```
(base)
```

Run `conda deactivate` again if you also want to leave the `base` environment.

1.11 Installing Packages

Before installing packages, first activate the correct environment:

```
conda activate python-course
```

The general syntax for installing a package is:

```
conda install package-name
```

For example, install NumPy with:

```
conda install numpy
```

Install several packages together with:

```
conda install numpy pandas matplotlib
```

For this course, the commonly used packages can be installed with:

```
conda install numpy pandas matplotlib jupyterlab
```

A package can also be installed into a specified environment without activating it:

```
conda install -n python-course numpy
```

However, beginners are encouraged to activate the environment first. This makes it easier to see which environment is being modified.

1.12 Installing a Specific Package Version

A specific version of a package can be installed using the following syntax:

```
conda install package-name=version
```

For example:

```
conda install numpy=2.2
```

Conda will automatically examine package dependencies and determine whether the requested version is compatible with the existing environment.

1.13 Installing Packages from a Channel

A Conda channel is a repository from which packages can be downloaded.

If a package is unavailable in the default channel, it may be available from the `conda-forge` channel:

```
conda install -c conda-forge package-name
```

For example:

```
conda install -c conda-forge openpyxl
```

The option `-c conda-forge` tells Conda to search for the package in the `conda-forge` channel.

1.14 Searching for a Package

To check whether a package is available through Conda, use:

```
conda search package-name
```

For example:

```
conda search numpy
```

This command displays available versions of the package.

1.15 Viewing Installed Packages

To list the packages installed in the current environment, run:

```
conda list
```

To list packages in a specified environment without activating it, run:

```
conda list -n python-course
```

To check whether a particular package is installed, its name can be added to the command:

```
conda list numpy
```

The output normally includes:

- the package name;
- the installed version;
- the build information;
- the package channel.

1.16 Testing an Installed Package

After installing a package, start Python:

```
python
```

Then import the package:

```
import numpy
print(numpy.**version**)
```

If no error message appears, the package has been imported successfully.

To leave the Python interactive environment, enter:

```
exit()
```

Alternatively, test the package directly from the command line:

```
python -c "import numpy; print(numpy.**version**)"
```

1.17 Updating Packages

To update one package in the current environment, run:

```
conda update package-name
```

For example:

```
conda update numpy
```

To update all packages in the current environment, run:

```
conda update --all
```

Updating all packages may change many dependencies. Therefore, it is advisable to inspect the proposed changes before confirming the operation.

1.18 Removing Packages

To remove a package from the current environment, run:

```
conda remove package-name
```

For example:

```
conda remove numpy
```

To remove a package from a specified environment, run:

```
conda remove -n python-course numpy
```

Conda will also display any dependent packages that must be removed.

1.19 Using Pip inside a Conda Environment

Some Python packages may not be available through Conda. In this case, they can be installed with `pip`.

First, activate the correct environment:

```
conda activate python-course
```

Then use:

```
python -m pip install package-name
```

For example:

```
python -m pip install requests
```

Using `python -m pip` is preferable to using `pip` alone because it makes clear which Python interpreter is running `pip`.

The recommended package installation order is:

1. activate the correct Conda environment;
2. try to install the package using `conda`;
3. use `pip` only when the package is unavailable through Conda;
4. avoid repeatedly switching between Conda and `pip` for the same package.

1.20 Installing and Starting JupyterLab

Activate the course environment:

```
conda activate python-course
```

Install JupyterLab:

```
conda install jupyterlab
```

Start JupyterLab with:

```
jupyter lab
```

The JupyterLab interface should open in a web browser.

The classic Jupyter Notebook can be installed and started with:

```
conda install notebook
```

```
jupyter notebook
```

To stop the Jupyter server, return to the command-line window and press:

Ctrl+C

Confirm the operation if the terminal asks whether the server should be stopped.

1.21 Registering a Jupyter Kernel

If the Conda environment does not appear in Jupyter, install `ipykernel`:

```
conda activate python-course
```

```
conda install ipykernel
```

Then register the environment as a Jupyter kernel:

```
python -m ipykernel install --user --name python-course --display-name "Python (python-course)"
```

After restarting JupyterLab, the kernel should appear as:

Python (python-course)

When creating a notebook, select this kernel to ensure that the notebook uses packages installed in the course environment.

1.22 Cloning an Environment

An existing environment can be copied using the `--clone` option.

For example:

```
conda create -n python-course-backup --clone python-course
```

This creates a new environment named `python-course-backup` based on the existing `python-course` environment.

Cloning is useful before performing major package updates or experiments.

1.23 Exporting an Environment

A Conda environment can be exported to a YAML file:

```
conda activate python-course
conda env export > environment.yml
```

The generated `environment.yml` file records the environment name, channels, packages, and package versions.

For a more portable file containing mainly the packages explicitly requested by the user, run:

```
conda env export --from-history > environment.yml
```

The `--from-history` option normally produces a shorter file and reduces operating-system-specific dependencies.

1.24 Creating an Environment from a YAML File

If an `environment.yml` file is available, recreate the environment with:

```
conda env create -f environment.yml
```

The environment name is normally read from the first part of the YAML file:

```
name: python-course
```

After the environment has been created, activate it:

```
conda activate python-course
```

To update an existing environment using the YAML file, run:

```
conda env update -f environment.yml --prune
```

The `--prune` option removes packages that are no longer included in the environment specification.

1.25 Removing an Environment

Before removing an environment, make sure that it is not active:

```
conda deactivate
```

Then remove the environment and all packages installed in it:

```
conda remove -n python-course --all
```

An equivalent command is:

```
conda env remove -n python-course
```

Finally, verify that the environment has been removed:

```
conda env list
```

Removing an environment does not normally affect other Conda environments.

1.26 Cleaning the Conda Cache

Conda stores downloaded packages and temporary files in a local cache.

To see what can be removed, run:

```
conda clean --all --dry-run
```

The `--dry-run` option previews the operation without deleting files.

To perform the cleanup, run:

```
conda clean --all
```

Read the displayed information carefully before confirming the operation.

1.27 Controlling Automatic Activation of Base

By default, Conda may activate the `base` environment whenever a terminal is opened.

To disable automatic activation, run:

```
conda config --set auto_activate_base false
```

Close and reopen the terminal after running the command.

To enable automatic activation again, run:

```
conda config --set auto_activate_base true
```

Even when automatic activation is disabled, the base environment can still be activated manually:

```
conda activate base
```

1.28 Initializing Conda for a Terminal

If `conda activate` does not work in a normal terminal, Conda may need to be initialized:

```
conda init
```

Conda can also be initialized for a particular shell.

For Windows PowerShell:

```
conda init powershell
```

For the Windows Command Prompt:

```
conda init cmd.exe
```

For Bash on macOS or Linux:

```
conda init bash
```

For Zsh on macOS:

```
conda init zsh
```

After running `conda init`, close and reopen the terminal.

1.29 Common Problems

Problem 1: Conda Is Not Recognized

A possible error message is:

```
'conda' is not recognized as an internal or external command
```

Possible solutions include:

1. On Windows, open **Anaconda Prompt** instead of the normal Command Prompt.
2. Check whether Anaconda or Miniconda has been installed.
3. Run `conda init` from Anaconda Prompt.
4. Close and reopen the terminal.
5. Restart the computer if the terminal configuration has not been refreshed.

Problem 2: The Wrong Python Interpreter Is Used

First, check the active environment:

```
conda env list
```

Then check the Python interpreter location:

```
python -c "import sys; print(sys.executable)"
```

If the location does not contain the expected environment name, activate the correct environment:

```
conda activate python-course
```

Problem 3: A Package Cannot Be Imported

For example:

```
ModuleNotFoundError: No module named 'pandas'
```

Check the following:

1. Is the correct environment active?
2. Is the package installed in this environment?
3. Is Jupyter using the correct kernel?

Useful commands are:

```
conda env list
conda list pandas
python -c "import sys; print(sys.executable)"
```

If the package is not installed, run:

```
conda install pandas
```

Problem 4: PackagesNotFoundError

A possible error message is:

`PackagesNotFoundError: The following packages are not available`

First, check the package name:

```
conda search package-name
```

Then try the `conda-forge` channel:

```
conda install -c conda-forge package-name
```

If the package is still unavailable, activate the environment and try pip:

```
python -m pip install package-name
```

Problem 5: The Environment Is Missing from Jupyter

Install and register the kernel again:

```
conda activate python-course
```

```
conda install ipykernel
```

```
python -m ipykernel install --user --name python-course --display-name "Python (python-course)"
```

Then close and restart JupyterLab.

1.30 Recommended Workflow for This Course

Students are encouraged to follow this workflow:

1. Open Anaconda Prompt or a terminal.
2. Check the existing environments:

```
conda env list
```

““

3. Activate the course environment: ““

```
conda activate python-course
```

““

4. Confirm the Python interpreter: ““

```
python --version
```

```
python -c "import sys; print(sys.executable)"
```

““

5. Start JupyterLab: ““

```
jupyter lab
```

““

6. Complete and save the programming exercises.
7. Stop JupyterLab with `Ctrl+C`.
8. Deactivate the environment: ““

```
conda deactivate
```

1.31 Summary of Common Conda Commands

Command	Purpose
<code>conda --version</code>	Display the installed Conda version.
<code>conda info</code>	Display detailed information about Conda.
<code>conda update -n base conda</code>	Update Conda itself.
<code>conda create -n python-course python=3.12</code>	Create a new environment with Python 3.12.
<code>conda env list</code>	List all Conda environments.
<code>conda activate python-course</code>	Activate the course environment.
<code>conda deactivate</code>	Leave the current environment.
<code>conda install numpy</code>	Install a package in the active environment.
<code>conda install numpy pandas matplotlib</code>	Install several packages together.
<code>conda install -c conda-forge openpyxl</code>	Install a package from the conda-forge channel.
<code>conda search numpy</code>	Search for available package versions.
<code>conda list</code>	List packages in the active environment.
<code>conda list -n python-course</code>	List packages in a specified environment.
<code>conda update numpy</code>	Update a package.
<code>conda remove numpy</code>	Remove a package.
<code>python -m pip install requests</code>	Install a package using pip in the active environment.
<code>conda create -n backup --clone python-course</code>	Clone an existing environment.
<code>conda env export --from-history > environment.yml</code>	Export a portable environment specification.
<code>conda env create -f environment.yml</code>	Create an environment from a YAML file.
<code>conda remove -n python-course --all</code>	Remove an environment and all its packages.
<code>conda clean --all</code>	Remove unused package caches and temporary files.
<code>conda init</code>	Initialize Conda for the current shell.
<code>jupyter lab</code>	Start JupyterLab.

1.32 Important Reminders

- Always check the environment name before installing a package.
- Create a separate environment for each course or project.
- Avoid installing all project packages in the **base** environment.
- Use Conda first and pip only when necessary.
- In Jupyter, confirm that the selected kernel belongs to the correct environment.
- Export the environment file when sharing a project with other users.
- Do not delete an environment before saving important code and files.

2 Assignment: Conda Environment Practice

2.1 Objectives

After completing this assignment, students should be able to:

- install Anaconda successfully;
- create and activate a Conda virtual environment;
- check the Python and Conda versions;
- install and import Python packages;
- run a simple Python program in the correct environment;
- export a Conda environment configuration.

2.2 Submission Requirements

Please submit a single PDF document containing:

- your name and student ID;
- the commands used for each task;
- screenshots of the required results;
- short answers to the questions;
- the Python code used in the assignment.

The environment name should contain your student ID. For example:

```
python-20260001
```

Make sure that the environment name and command output are clearly visible in each screenshot.

2.3 Task 1: Install Anaconda and Create an Environment

Install Anaconda on your computer. After the installation is complete, open Anaconda Prompt or a terminal and check whether Conda has been installed successfully.

Run the following commands:

```
conda --version
conda info
```

Create a new Conda environment containing your student ID. For example:

```
conda create -n python-20260001 python=3.12
```

Activate the environment:

```
conda activate python-20260001
```

List all available Conda environments:

```
conda env list
```

Submission:

1. Take a screenshot showing the output of `conda --version`.
2. Take a screenshot showing the list of Conda environments.
3. The screenshot must clearly show that your newly created environment is active.

The active environment is normally marked by an asterisk `*`, and its name also appears at the beginning of the command line.

2.4 Task 2: Display Personal and Environment Information

Write a Python program that prints the following information:

- your name;
- your student ID;
- the current Python version;
- the name of the current Conda environment.

You may use a Python script, Jupyter Notebook, or the Python interactive interpreter.

A sample Python program is shown below:

```
import os
import sys

name = "Your Name"
student_id = "Your Student ID"

print("Name:", name)
print("Student ID:", student_id)
print("Python Version:", sys.version)
print("Conda Environment:", os.environ.get("CONDA_DEFAULT_ENV"))
```

The expected output is similar to:

```
Name: Zhang San
Student ID: 20260001
Python Version: 3.12.8
Conda Environment: python-20260001
```

Because `sys.version` may display additional system information, you may use the following code to obtain a shorter Python version:

```
print("Python Version:", sys.version.split()[0])
```

Submission:

1. Submit your complete Python code.
2. Take a screenshot showing both the code and its output.
3. Make sure that the displayed environment name is the environment created in Task 1.

2.5 Task 3: Check the Python Interpreter

When several Python environments exist on the same computer, it is important to check which Python interpreter is currently being used.

Run:

```
python -c "import sys; print(sys.executable)"
```

Then run:

```
python --version
```

Questions:

1. What is the path of your current Python interpreter?
2. Does the path contain the name of the environment created in Task 1?
3. Why is it important to check the Python interpreter before installing packages?

Submission:

Take a screenshot showing the interpreter path and Python version.

2.6 Task 4: Install and Test Python Packages

Activate your environment and install the following packages:

```
conda install numpy pandas matplotlib
```

Use the following command to check whether the packages have been installed:

```
conda list
```

Next, create a Python program that imports the three packages and prints their versions:

```
import numpy
import pandas
import matplotlib

print("NumPy Version:", numpy.__version__)
print("Pandas Version:", pandas.__version__)
print("Matplotlib Version:", matplotlib.__version__)
```

Submission:

1. Take a screenshot showing that the three packages are installed.
2. Take a screenshot showing that the program runs without errors.
3. Submit the complete Python code.

2.7 Task 5: Create and Run a Python File

Create a file named:

```
student_information.py
```

The program should print your personal and environment information in the following format:

```
#
```

```
# Student Information
```

```
Name: Zhang San
Student ID: 20260001
Python Version: 3.12.8
Conda Environment: python-20260001
=====
```

Run the program from the command line:

```
python student_information.py
```

Submission:

1. Submit the `student_information.py` file.
2. Take a screenshot showing the command used to run the file.
3. The screenshot should include the complete program output.

2.8 Task 6: Export the Conda Environment

Export the packages explicitly installed in your environment:

```
conda env export --from-history > environment.yml
```

Open the generated `environment.yml` file and examine its contents.

A possible file structure is:

```
name: python-20260001
channels:
```

```
* defaults
  dependencies:
* python=3.12
* numpy
* pandas
* matplotlib
```

The actual content may be different depending on your operating system and package versions.

Questions:

1. What information is stored in the `environment.yml` file?
2. What command can another student use to recreate your environment?
3. Why is an environment configuration file useful when working in a team?

The command for recreating an environment is:

```
conda env create -f environment.yml
```

Submission:

Submit the generated `environment.yml` file and a screenshot of its contents.

2.9 Task 7: Deactivate and Reactivate the Environment

Deactivate the current environment:

```
conda deactivate
```

Then activate it again:

```
conda activate python-20260001
```

Run the personal information program again:

```
python student_information.py
```

Questions:

1. What changes at the beginning of the command line after the environment is activated?
2. Can the program still display the correct environment name?
3. What is the difference between creating an environment and activating an environment?

2.10 Optional Challenge: Compare Two Environments

This task is optional.

Create another environment with a different Python version. For example:

```
conda create -n python-test python=3.11
```

Activate the new environment:

```
conda activate python-test
```

Run:

```
python --version  
python -c "import sys; print(sys.executable)"
```

Then switch back to your original environment:

```
conda activate python-20260001
```

Questions:

1. Do the two environments use the same Python version?
2. Do the two environments use the same Python interpreter path?
3. Are packages installed in one environment automatically available in the other environment?
4. Why can virtual environments reduce package conflicts?

2.11 Checklist

Before submitting the assignment, confirm that you have completed the following items:

- ☐ Anaconda has been installed successfully.
- ☐ A virtual environment containing your student ID has been created.
- ☐ The environment has been activated successfully.
- ☐ Your name and student ID are displayed correctly.
- ☐ The Python version is displayed correctly.
- ☐ The Conda environment name is displayed correctly.
- ☐ NumPy, pandas, and Matplotlib have been installed.
- ☐ The Python program runs without errors.
- ☐ The environment has been exported to an `environment.yml` file.
- ☐ All required screenshots are clear and complete.

2.12 Grading Criteria

Item	Points
Successful installation of Anaconda and verification of Conda	15
Creation and activation of the virtual environment	20
Correct display of name, student ID, Python version, and environment name	20
Correct installation and import of Python packages	15
Successful execution of the Python script	15
Environment export and answers to the questions	10
Clear screenshots and complete submission	5
Total	100

2.13 Important Notes

- Replace all sample names and student IDs with your own information.
- Do not copy another student's screenshots or environment name.
- Screenshots should show both the command and its output.
- Do not submit screenshots containing only part of the result.
- The assignment should be completed in the environment created by you.
- Save all code and screenshots before deleting or modifying an environment.

Assignment Requirement and Deadline: Tasks 1–2 are compulsory and must be submitted. Tasks 3–7 are optional and are provided for further practice and understanding. The deadline is before the second class.